pg_repack for PostgreSQL

Why it is used, and how to install, configure and run it

pg_repack is an open-source PostgreSQL extension and command-line tool that removes bloat from tables and indexes and can restore the physical order of a table, **while the table stays available for reads and writes**. This guide explains the problem it solves, how it works internally, and gives a complete step-by-step procedure.

Contents

- 1. The problem: table and index bloat
- 2. Why pg_repack is used (compared with VACUUM, VACUUM FULL, CLUSTER)
- 3. How pg_repack works internally
- 4. Requirements and limitations
- 5. Install pg_repack (package or source)
- 6. Enable the extension in a database
- 7. Detect bloat before repacking
- 8. Run pg_repack: step-by-step procedures
- 9. Command-line options reference
- 10. Monitoring, failures and cleanup
- 11. Upgrade and uninstall
- 12. Best practices checklist and quick reference

1. The problem: table and index bloat

PostgreSQL uses MVCC (multi-version concurrency control). When a row is updated or deleted, the old version is not removed immediately; it becomes a **dead tuple**. Regular VACUUM (and autovacuum) marks the space occupied by dead tuples as reusable inside the same table file, but it **does not normally shrink the file** or hand space back to the operating system.

Over time, tables and indexes that see heavy UPDATE and DELETE activity accumulate unused space. This is called **bloat**. Its effects are:

- **Wasted disk space:** the table or index file is much larger than the live data inside it.
- **Slower queries:** sequential scans read many half-empty pages, and bloated indexes mean more pages to traverse.
- **Lower cache efficiency:** shared buffers and the OS cache fill up with pages that contain little useful data.
- **Longer backups, vacuum and replication times** because more data must be processed and shipped.

2. Why pg_repack is used

To reclaim bloated space, you need to rewrite the table into a new compact file. PostgreSQL has built-in ways to do this, but they take a lock that blocks the application. pg_repack does the same rewrite **online**.

Method	Reclaims space to OS?	Locking during operation	Typical use
VACUUM / autovacuum	Mostly no (only empty pages at the end of the file)	No blocking of reads or writes	Routine maintenance; prevents bloat from growing
VACUUM FULL	Yes	ACCESS EXCLUSIVE lock for the whole rewrite: blocks reads and writes	Maintenance windows with downtime
CLUSTER	Yes, and orders rows by an index	ACCESS EXCLUSIVE lock for the whole rewrite	Maintenance windows with downtime
REINDEX	Index only	Blocks writes (REINDEX CONCURRENTLY avoids this on PG 12+)	Index bloat or corruption
pg_repack	Yes	Only very short exclusive locks at start and end; normal reads and writes continue	Production systems that cannot afford downtime

Main uses of pg_repack

- **Remove table bloat** online (like VACUUM FULL without the long outage).
- **Remove index bloat** for all indexes of a table, or one specific index.
- **Restore clustering order** of a table by a chosen index or by ORDER BY columns (like CLUSTER, but online).
- **Move a table or its indexes to another tablespace** online.

3. How pg_repack works internally

For each table you ask it to repack, pg_repack follows these steps:

- **1. Take a brief exclusive lock** on the table and create a **log table** plus a **trigger** on the original table. From now on every INSERT, UPDATE and DELETE is also recorded in the log table.
- **2. Copy the data** into a new table (optionally in the requested order) without blocking users. The copy is a consistent snapshot, so it contains only live rows, with no dead space.
- **3. Build all indexes** on the new table.
- **4. Replay the log:** changes that happened during the copy are applied to the new table.
- **5. Take a brief ACCESS EXCLUSIVE lock**, apply the last logged changes and swap the old and new tables (their storage files).
- **6. Drop the old table**, remove the trigger and log table, and run ANALYZE on the new table (unless disabled).

The only moments where applications may be briefly blocked are step 1 and step 5. If a long-running transaction holds a conflicting lock, pg_repack waits, and after a timeout it can cancel (or with `-D` not cancel) the blocking sessions.

Disk space note: because a full copy of the table and its indexes exists alongside the original until the swap, you need roughly **twice the size of the table plus indexes** as free space, plus extra WAL.

4. Requirements and limitations

Requirements

- The table must have a **PRIMARY KEY** or a **UNIQUE index on NOT NULL columns**. Without one, pg_repack refuses to run on that table.
- You must run it as a **superuser** (or use `--no-superuser-check` when your platform provides a privileged non-superuser role).
- The **pg_repack extension must be created** in each database you want to repack.
- The **pg_repack client version must match** the extension version installed in the database.
- Enough free disk space (about 2x the table and index size) and available WAL space.
- Run against the **primary server**, not a read-only standby.

Limitations

- Temporary tables cannot be repacked.
- Tables with GiST indexes cannot be clustered by that index, and tables using certain special index types need care. Check the release notes of your version.
- System catalog tables cannot be repacked.
- It generates a lot of WAL and I/O, which can increase replication lag on busy systems.
- DDL (ALTER TABLE, etc.) on the table during repack is blocked or may cause the repack to fail.

5. Install pg_repack

Install it on the **database server** (the extension files) and on any machine from which you run the `pg_repack` client (often the same server). Replace `16` with your PostgreSQL major version.

Step 5.1: Check your PostgreSQL version

```
sudo -u postgres psql -c "SELECT version();"
```

Option A: Package manager (recommended)

```
# Debian / Ubuntu (PGDG apt repository)
sudo apt update
sudo apt install postgresql-16-repack

# RHEL / CentOS / Rocky / Alma (PGDG yum repository)
sudo dnf install pg_repack_16
```

If the package is not found, enable the official PostgreSQL (PGDG) repository first.

Option B: Build from source

```
# 1. Install build dependencies
# Debian/Ubuntu:
sudo apt install build-essential postgresql-server-dev-16 \
     libssl-dev zlib1g-dev libreadline-dev git
# RHEL family:
sudo dnf install gcc make postgresql16-devel openssl-devel \
     zlib-devel readline-devel git

# 2. Make pg_config for your version visible
export PATH=/usr/lib/postgresql/16/bin:$PATH     # Debian/Ubuntu
# export PATH=/usr/pgsql-16/bin:$PATH             # RHEL family

# 3. Get the source and build
git clone https://github.com/reorg/pg_repack.git
cd pg_repack
make
sudo make install
```

Step 5.2: Verify the client is installed

```
pg_repack --version
which pg_repack
```

6. Enable the extension in a database

The extension must be created in every database you plan to repack. No change to `shared_preload_libraries` and no restart is required.

```
sudo -u postgres psql -d mydb

mydb=# CREATE EXTENSION pg_repack;
mydb=# \dx pg_repack
mydb=# SELECT extversion FROM pg_extension WHERE extname='pg_repack';
```

Check that the version printed by `pg_repack --version` matches the extension version in the database. A mismatch gives an error such as *program 'pg_repack 1.x' does not match database library*; fix it by installing the matching package or running `ALTER EXTENSION pg_repack UPDATE;`.

Connection setup: pg_repack connects like psql. Use `-h`, `-p`, `-U`, `-d`, or the environment variables `PGHOST`, `PGPORT`, `PGUSER`, `PGDATABASE`, and `~/.pgpass` for non-interactive passwords.

Managed services: Amazon RDS/Aurora PostgreSQL and Google Cloud SQL support the extension, but you run the client from another host and use the provider's privileged role. Check the provider documentation for the supported version.

7. Detect bloat before repacking

Do not repack blindly. First confirm that the table is actually bloated. Use the `pgstattuple` extension (shipped in contrib).

Step 7.1: Enable pgstattuple

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;
```

Step 7.2: Measure a table

```
SELECT table_len/1024/1024 AS size_mb,
       tuple_percent, dead_tuple_percent, free_percent
FROM pgstattuple('public.orders');
```

A high `free_percent` plus `dead_tuple_percent` (for example over 30 to 40 percent on a large table) indicates significant bloat. For very large tables use `pgstattuple_approx('public.orders')`, which is faster.

Step 7.3: Measure an index

```
SELECT avg_leaf_density, leaf_fragmentation
FROM pgstatindex('public.orders_pkey');
```

A low `avg_leaf_density` (well below 90 percent for a normal B-tree) suggests index bloat.

Step 7.4: Quick look at dead tuples and table sizes

```
SELECT relname, n_live_tup, n_dead_tup,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 20;
```

Step 7.5: Check the table has a usable key

```
SELECT conname, contype FROM pg_constraint
WHERE conrelid = 'public.orders'::regclass AND contype IN ('p','u');
```

8. Run pg_repack: step-by-step procedures

Step 8.1: Pre-checks

- Confirm free disk space is at least 2x the table plus index size: `SELECT pg_size_pretty(pg_total_relation_size('public.orders'));`
- Check for long-running transactions that could delay the lock steps:

```
SELECT pid, usename, state, now()-xact_start AS xact_age, left(query,60)
FROM pg_stat_activity
WHERE xact_start IS NOT NULL ORDER BY xact_start LIMIT 10;
```

- Verify replication is healthy and has headroom for extra WAL.
- Take a backup, or make sure a recent one exists.
- Choose a low-traffic period.

Step 8.2: Dry run

```
pg_repack -h localhost -U postgres -d mydb -t public.orders --dry-run
```

A dry run (`-N`) connects and checks the table without changing anything.

Step 8.3: Repack one table

```
pg_repack -h localhost -U postgres -d mydb -t public.orders -e
```

`-e` echoes the SQL pg_repack runs, which is useful for learning and troubleshooting.

Step 8.4: Other common scenarios

```
# Repack several tables
pg_repack -d mydb -t public.orders -t public.order_items

# Repack only the indexes of a table
pg_repack -d mydb -t public.orders --only-indexes

# Repack one specific index
pg_repack -d mydb --index public.orders_customer_idx

# Reorder a table by columns (clustering)
pg_repack -d mydb -t public.orders -o "created_at DESC"

# Repack without reordering (skip CLUSTER ordering, faster)
pg_repack -d mydb -t public.orders --no-order

# Move table to another tablespace (and optionally its indexes)
pg_repack -d mydb -t public.orders -s fast_ssd --moveidx fast_ssd

# Repack every table in a schema
pg_repack -d mydb --schema=sales

# Repack all tables in all databases that have the extension
pg_repack --all

# Use 4 parallel jobs to rebuild indexes faster
pg_repack -d mydb -t public.orders -j 4
```

Step 8.5: Run it in the background and keep a log

```
nohup pg_repack -d mydb -t public.orders -e > repack_orders.log 2>&1 &
tail -f repack_orders.log
```

Step 8.6: Verify the result

```
SELECT pg_size_pretty(pg_total_relation_size('public.orders'));
SELECT * FROM pgstattuple('public.orders');
-- statistics are refreshed automatically unless --no-analyze was used
```

Compare the size and `free_percent` with the values you recorded in step 7. The table should be much smaller and compact.

9. Command-line options reference

Option	Description
-d, --dbname	Database to connect to.
-h, -p, -U, -w, -W	Host, port, user, never prompt for password, force password prompt.
-a, --all	Repack all databases that have the extension installed.
-t, --table=TABLE	Repack the given table (repeat for several tables).
-c, --schema=SCHEMA	Repack all tables in the given schema.
-I, --parent-table	Repack the table and its inheritance children or partitions.
-x, --only-indexes	Repack only the indexes of the given table.
-i, --index=INDEX	Repack the specified index only.
-o, --order-by=COLUMNS	Order rows by the given columns while rebuilding.
-n, --no-order	Do not order rows; plain rewrite (like VACUUM FULL).
-s, --tablespace=NAME	Move the table to this tablespace.
-S, --moveidx	Also move indexes to the tablespace given with -s.
-j, --jobs=N	Number of parallel connections used to rebuild indexes.
-T, --wait-timeout=SEC	Seconds to wait for conflicting locks before acting (default 60).
-D, --no-kill-backend	Do not cancel blocking queries when the timeout is reached; abort instead.
-Z, --no-analyze	Skip ANALYZE after repack.
-k, --no-superuser-check	Skip the superuser check (managed services).
-N, --dry-run	Show what would happen without changing anything.
-e, --echo	Print the SQL commands being executed.
-E, --elevel=LEVEL	Log level: DEBUG, INFO, NOTICE, WARNING, ERROR.

Run `pg_repack --help` to see the options supported by your installed version.

10. Monitoring, failures and cleanup

Monitor progress

```
SELECT pid, state, wait_event_type, wait_event, left(query,80)
FROM pg_stat_activity WHERE application_name LIKE 'pg_repack%';

SELECT client_addr, state, replay_lag FROM pg_stat_replication;
```

Common errors and fixes

Symptom	Cause and fix
relation ... must have a primary key or not-null unique keys	Add a PRIMARY KEY or a UNIQUE index on NOT NULL columns, then retry.
pg_repack ... does not match database library	Client and extension versions differ. Install the matching version or ALTER EXTENSION pg_repack UPDATE.
permission denied / must be superuser	Run as superuser or use --no-superuser-check with an appropriate role on managed services.
extension 'pg_repack' not installed	Run CREATE EXTENSION pg_repack; in that database.
Waits a long time at start or end	A long transaction holds a lock. Find it in pg_stat_activity, end it, or let pg_repack cancel it after the timeout.
No space left on device	Not enough free space for the second copy. Free space first, or repack smaller tables one at a time.
Replication lag grows	Heavy WAL volume. Repack off-peak, one table at a time, or use fewer jobs.

Cleaning up after a failed or interrupted run

If pg_repack is killed, it can leave a trigger, a log table and a temporary copy in the `repack` schema. Try re-running once; if leftovers remain, remove them by recreating the extension:

```
DROP EXTENSION pg_repack CASCADE;
CREATE EXTENSION pg_repack;
```

Before dropping, make sure no pg_repack process is still running, and check that the original table still has no stray `repack_trigger` (`\d public.orders`).

11. Upgrade and uninstall

Upgrade

```
# 1. Upgrade the OS package / rebuild from newer source
sudo apt install --only-upgrade postgresql-16-repack
# 2. Update the extension in each database
psql -d mydb -c "ALTER EXTENSION pg_repack UPDATE;"
```

If the update path is not available for your versions, drop and recreate the extension (safe when no repack is running).

Uninstall

```
psql -d mydb -c "DROP EXTENSION pg_repack;"
sudo apt remove postgresql-16-repack      # or: sudo dnf remove pg_repack_16
```

12. Best practices checklist and quick reference

- Measure bloat first with pgstattuple; repack only tables that need it.
- Tune autovacuum so bloat does not build up again (lower `autovacuum_vacuum_scale_factor` on busy tables).
- Run during low traffic and avoid DDL on the table while it runs.

- Repack tables one at a time, largest ones individually, and watch disk, I/O and replication lag.
- Ensure free space of at least 2x table plus indexes, plus WAL headroom.
- Look for long-running or idle-in-transaction sessions before starting.
- Test on a staging copy first and keep a recent backup.
- Use `-e` and log output to a file for auditing.
- After repack, confirm the new size and that queries and statistics look healthy.

Minimum end-to-end recap

```
# 1. Install
sudo apt install postgresql-16-repack

# 2. Enable in the database
psql -d mydb -c "CREATE EXTENSION pg_repack;"

# 3. Check bloat
psql -d mydb -c "CREATE EXTENSION IF NOT EXISTS pgstattuple;"
psql -d mydb -c "SELECT * FROM pgstattuple('public.orders');"

# 4. Dry run, then repack
pg_repack -d mydb -t public.orders --dry-run
pg_repack -d mydb -t public.orders -e

# 5. Verify size
psql -d mydb -c "SELECT pg_size_pretty(pg_total_relation_size('public.orders'));"
```

Official documentation and source: `https://github.com/reorg/pg_repack` and `https://reorg.github.io/pg_repack/`